Robótica con Arduino y micro:bit

Nombre: _____ Fecha: _____

Qué vas a aprender

- Conocer las placas programables Arduino y micro:bit y sus partes
- Montar circuitos sencillos con LED, pulsador y resistencia en una placa de prototipos
- Programar las funciones básicas setup y loop y la lectura y escritura de pines
- Programar la micro:bit con MakeCode usando sus sensores integrados

Para pensar

¿Cómo puede una placa del tamaño de una tarjeta de crédito controlar un robot, una estación meteorológica o una instalación artística?

Explicación

1.1 Placas programables

Un **microcontrolador** es un chip que contiene procesador, memoria y entradas y salidas. A diferencia de un ordenador, ejecuta un único programa una y otra vez, desde que se enciende hasta que se apaga.

- **Arduino** nació en Italia en 2005 como proyecto de **hardware libre**: sus diseños son públicos y cualquiera puede fabricar placas compatibles. Se programa en un lenguaje basado en C++ o con bloques.
- **micro:bit** fue creada por la BBC en 2015 para que los estudiantes aprendieran a programar. Lleva sensores y luces ya integrados y se programa con bloques, JavaScript o Python.

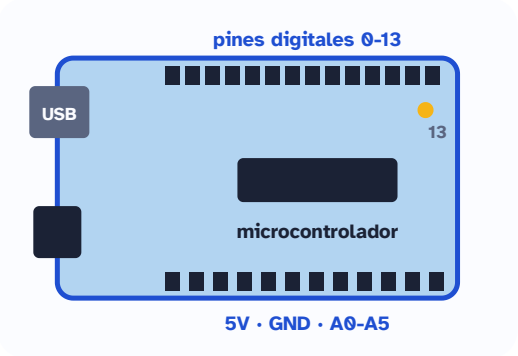


1.2 La placa Arduino UNO

Es el modelo más usado en las aulas. Sus partes principales:

Parte	Función
Microcontrolador (ATmega328P)	el «cerebro»; guarda y ejecuta el programa
Puerto USB	conecta al ordenador para cargar el programa y alimentar la placa
Conector de alimentación	pila o fuente de 7-12 V para funcionar sin ordenador
Pines digitales 0-13	entradas o salidas de 0 o 5 V; algunos (~) imitan valores intermedios (PWM)

Parte	Función
Pines analógicos A0-A5	leen tensiones de 0 a 5 V como números de 0 a 1023
Pines 5V, 3.3V y GND	alimentación para sensores; GND es el negativo (tierra)
LED integrado (pin 13)	permite probar sin montar nada



1.3 La protoboard y el primer circuito

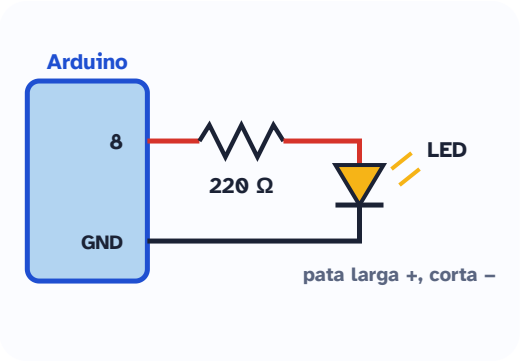
La **placa de prototipos** (*protoboard*) permite montar circuitos sin soldar. Sus agujeros están conectados por dentro: en las **columnas centrales**, de cinco en cinco; en los **carriles laterales**, a lo largo (para + y –).

Para encender un LED desde Arduino:

1. Cable del **pin 8** a una **resistencia de 220 Ω**.
2. La resistencia, a la **pata larga** del LED (ánodo, +).
3. La **pata corta** (cátodo, –), a **GND**.

La resistencia es imprescindible: limita la corriente para que el LED no se queme. Monta siempre con la placa **desconectada** y revisa antes de enchufar.

Antes de conectar la placa al ordenador conviene revisar el montaje: que el LED tenga su resistencia (220 Ω es un valor habitual a 5 V), que la pata larga vaya hacia el pin y la corta hacia GND, que ningún cable una directamente 5 V con GND (sería un cortocircuito) y que cada componente tenga sus patas en columnas distintas. Cualquier cambio en el circuito se hace con la placa desconectada.



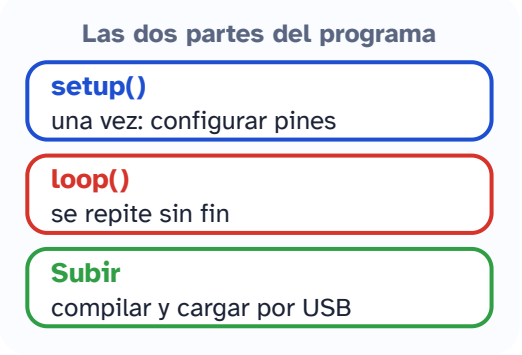
2.1 Programar Arduino: setup y loop

Un programa de Arduino (llamado **sketch**) tiene siempre dos partes:

- `setup()`: se ejecuta una sola vez al encender. Se configuran los pines.
- `loop()`: se repite sin fin. Aquí va lo que la placa hace continuamente.

```
void setup() {
  pinMode(8, OUTPUT);     // el pin 8 será una salida
}
void loop() {
  digitalWrite(8, HIGH);  // enciende el LED (5 V)
  delay(500);             // espera 500 ms
  digitalWrite(8, LOW);   // apaga el LED (0 V)
  delay(500);
}
```

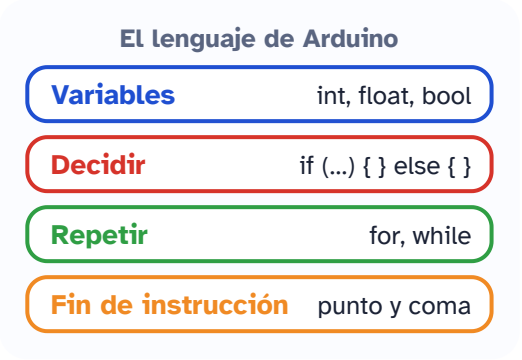
Este es el famoso «Blink»: el LED parpadea una vez por segundo. Se escribe en el **Arduino IDE** y se carga con el botón «Subir». También hay entornos de bloques, como **Arduinoblocks** o mBlock.



2.2 Variables, condiciones y funciones en Arduino

Arduino se programa en un lenguaje basado en **C++**. Las variables se declaran indicando su tipo: **int** para enteros (int contador = 0;), **float** para decimales (float temperatura;), **bool** para verdadero o falso y **String** para textos. Es buena costumbre dar nombre a los pines con constantes al principio: const int LED_ROJO = 10; así, si se cambia la conexión, solo hay que modificar una línea.

Las estructuras de control son las mismas que en cualquier lenguaje: **if (condición) { … } else { … }** para decidir, **for (int i = 0; i < 5; i++) { … }** para repetir un número de veces y **while (condición) { … }** para repetir mientras se cumpla algo. Las comparaciones usan == (igual), != (distinto), <, >, <= y >=, y las condiciones se combinan con && (y) y || (o). Se pueden crear **funciones** propias, como void parpadear(int veces) { … }, para no repetir código. Cada instrucción termina en punto y coma y los bloques van entre llaves: olvidar uno de estos signos es el error más frecuente al compilar.

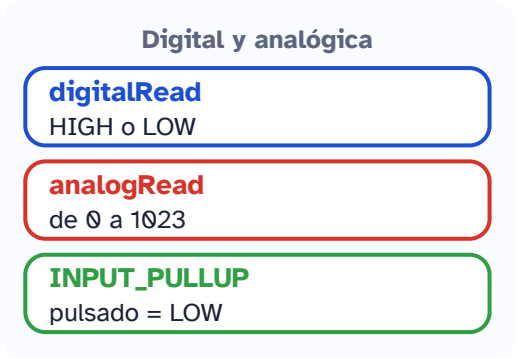


2.3 Leer entradas: pulsadores y sensores

Para leer un **pulsador**, el pin se configura como entrada y se usa `digitalRead`, que devuelve `HIGH` (pulsado) o `LOW`:

```
void setup() {
  pinMode(2, INPUT_PULLUP);   // pulsador en el pin 2
  pinMode(8, OUTPUT);
}
void loop() {
  if (digitalRead(2) == LOW) digitalWrite(8, HIGH);
  else digitalWrite(8, LOW);
}
```

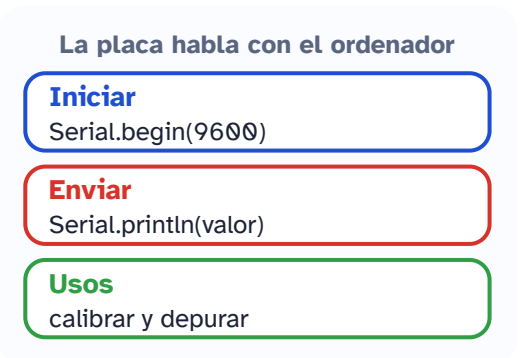
Con `INPUT_PULLUP`, el pin está a 5 V y baja a 0 V al pulsar, por eso pulsado es `LOW`. Para sensores analógicos se usa `analogRead(A0)`, que da un número de 0 a 1023. Con el **monitor serie** (`Serial.println`) se ven los valores en el ordenador: es la mejor herramienta para calibrar y depurar.



2.4 El monitor serie: ver lo que piensa la placa

La placa no tiene pantalla, pero puede enviar mensajes al ordenador por el cable USB. En setup() se inicia la comunicación con **Serial.begin(9600);** (9.600 bits por segundo) y en el programa se envían datos con **Serial.println(valor);**. En el entorno de Arduino, el **monitor serie** muestra esos mensajes en una ventana, y el **trazador serie** los dibuja como una gráfica que se actualiza en tiempo real.

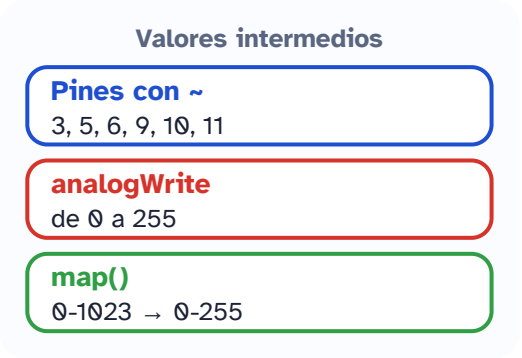
Es la herramienta principal para **calibrar sensores**: se imprime el valor de la LDR con luz y a oscuras para elegir el umbral, o la distancia que mide el ultrasonidos para comprobar que es correcta. También sirve para **depurar**: imprimir mensajes como «entrando en el if» o el valor de una variable permite saber por dónde pasa el programa y por qué no hace lo esperado. El monitor serie también funciona al revés: se pueden escribir órdenes desde el ordenador y la placa las lee con Serial.read().



2.5 Salidas analógicas: PWM y brillo

Los pines digitales solo dan 0 V o 5 V: encendido o apagado. ¿Cómo se consigue entonces que un LED brille a medias o que un motor gire despacio? Con la técnica **PWM** (modulación por ancho de pulsos): el pin se enciende y se apaga muy rápido, unas 500 veces por segundo, y lo que cambia es el porcentaje de tiempo que está encendido. Si está encendido la mitad del tiempo, el LED parece brillar a la mitad.

En Arduino UNO solo algunos pines admiten PWM: los marcados con el símbolo ~ (3, 5, 6, 9, 10 y 11). Se usa **analogWrite(pin, valor)**, con un valor de 0 (apagado) a 255 (máximo). Para hacer que un LED se encienda poco a poco se usa un bucle: for (int b = 0; b <= 255; b++) { analogWrite(9, b); delay(10); }. Combinando la lectura de un potenciómetro (0-1023) con analogWrite (0-255) se regula el brillo o la velocidad girando un mando; para pasar de una escala a otra se divide entre 4 o se usa la función **map()**.

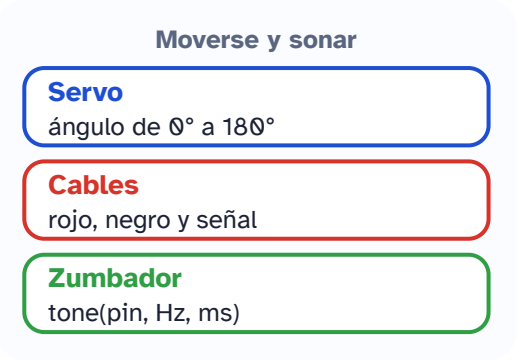


2.6 Servomotores y zumbadores

Un **servomotor** es un motor con una pequeña reductora y un circuito de control que lo lleva a un **ángulo concreto**, normalmente entre 0° y 180°, y lo mantiene ahí. Es ideal para barreras, brazos robóticos, timones o para girar un sensor. Tiene tres cables: alimentación (rojo), masa (marrón o negro) y señal (naranja o amarillo), que va a un pin PWM. En Arduino se usa la biblioteca Servo: #include <Servo.h>, se crea el objeto Servo barrera;, en setup() se indica el pin con barrera.attach(9); y se mueve con **barrera.write(90);**.

Un **zumbador** (buzzer) emite sonidos. Los activos pitan con solo darles tensión; los pasivos pueden tocar notas con la función **tone(pin, frecuencia, duración)**: tone(8, 440, 500) toca un la de 440 Hz durante medio segundo, y noTone(8) lo calla. Con una lista de frecuencias y un bucle se programan melodías y alarmas.

En la micro:bit, el servo se controla con «servo escribir pin P0 a 90» y el altavoz integrado toca notas con los bloques de música.

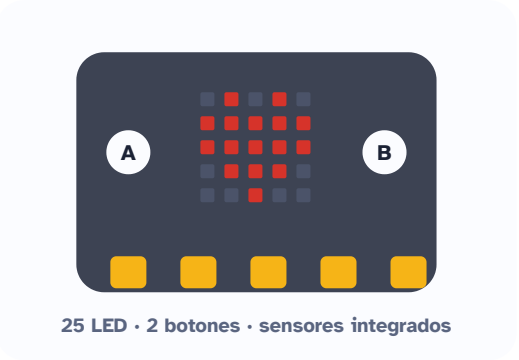


3.1 La placa micro:bit

La **micro:bit** trae casi todo integrado, así que se puede programar sin montar nada:

- Una **matriz de 25 LED** (5 × 5) que muestra iconos, texto y números, y que también mide la luz.
- **Dos botones** (A y B) y un logotipo táctil.
- **Acelerómetro** (detecta agitar e inclinar), **brújula** y sensor de **temperatura**.
- **Radio** y Bluetooth para comunicar varias placas.
- Altavoz y micrófono (en la versión 2).
- Un **conector de borde** con pines para añadir motores y sensores externos.

La micro:bit se alimenta por el cable USB o con un portapilas de dos pilas AAA, y su programa se guarda en la placa: al desconectarla del ordenador sigue funcionando. Sus pines grandes (0, 1, 2, 3V y GND) admiten pinzas de cocodrilo, lo que permite conectar sensores, LED o servos sin soldar ni usar protoboard.

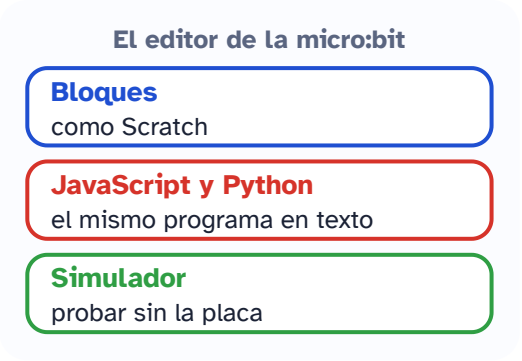


3.2 Programar la micro:bit con MakeCode

MakeCode es el editor web de Microsoft para la micro:bit. Funciona con bloques (como Scratch) y permite ver el mismo programa en JavaScript o Python. Incluye un **simulador** para probar sin la placa.

- **al iniciar**: equivale al `setup()` de Arduino.
- **para siempre**: equivale al `loop()`.
- Eventos: **al presionar el botón A**, **al agitar**, **al recibir por radio**...

Ejemplo, un dado electrónico: «al agitar → mostrar número (número al azar entre 1 y 6)». Para pasar el programa, se descarga un archivo `.hex` y se arrastra a la micro:bit, que el ordenador ve como una memoria USB.



3.3 Proyectos conectados: la radio de la micro:bit

Una de las funciones más interesantes de la micro:bit es su **radio**: varias placas pueden enviarse números, textos o pares nombre-valor sin cables, a unos metros de distancia. Todas deben estar en el mismo **grupo** («radio establecer grupo 1»). Una placa envía con «radio enviar número 5» y las demás reaccionan con el evento «al recibir radio número».

Con la radio se pueden hacer proyectos muy variados: un mando a distancia para un robot (una placa en la mano, que envía la inclinación, y otra en el robot que mueve los motores), un sistema de votación en clase, mensajes cifrados entre compañeros, un juego de «patata caliente» o una red de sensores que mide la temperatura en varias aulas y la envía a una placa central conectada al ordenador. También la intensidad de la señal recibida sirve para estimar si otra placa está cerca o lejos, como en un juego de búsqueda del tesoro. Para conectar con el móvil, la micro:bit usa Bluetooth.



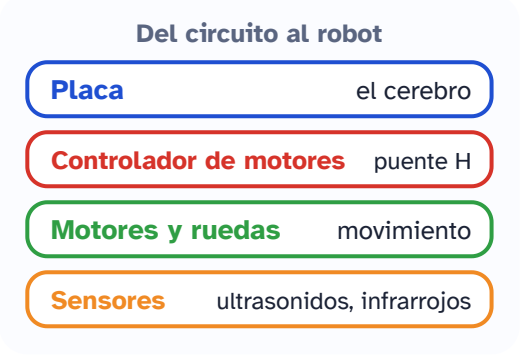
4.1 Del circuito al robot

Con una placa, dos **motores** con ruedas, un **controlador de motores**, una batería y uno o dos sensores ya tenemos un robot móvil:

- **Robot que esquiva**: ultrasonidos delante; si hay un obstáculo cerca, gira.
- **Robot siguelíneas**: dos sensores infrarrojos mirando al suelo; si uno ve negro, corrige hacia ese lado.
- **Robot teledirigido**: una micro:bit en la mano envía por radio la inclinación y otra en el robot mueve los motores.

Recomendaciones de seguridad: alimentar los motores aparte, no cortocircuitar la batería y desconectarla al modificar el montaje.

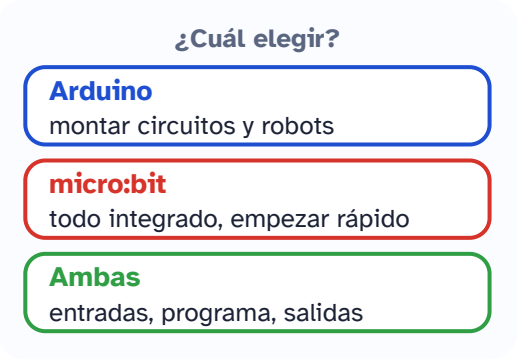
Un robot de dos ruedas necesita un **controlador de motores** (puente H, como el L298N o el L9110), porque los pines solo dan unos miliamperios y los motores necesitan mucho más. El controlador recibe órdenes de la placa (dos pines por motor: adelante o atrás) y toma la energía de una batería aparte. Para regular la velocidad se usa una salida PWM. Es importante unir las masas (GND) de la placa y de la batería de los motores.



4.2 Arduino o micro:bit: ¿cuál elegir?

	Arduino UNO	micro:bit
Programación	C++ (IDE) o bloques	bloques, JavaScript, Python
Sensores integrados	ninguno (solo un LED)	luz, temperatura, acelerómetro, brújula, botones
Montaje	protoboard y componentes	casi nada; pinzas de cocodrilo
Ventaja	muy versátil, enorme comunidad, hardware libre	empezar en minutos, ideal para aprender

Las dos comparten la misma lógica: leer entradas, decidir con un programa y controlar salidas. Hay además placas como **ESP32** (con wifi) o **Raspberry Pi**, que es ya un ordenador completo.



Robótica con placas programables

Microcontrolador	chip que ejecuta un programa sin parar
Arduino	pines 0-13 y A0-A5 · setup() y loop()
micro:bit	25 LED · botones · acelerómetro · MakeCode
Circuito LED	pin → 220 Ω → LED (pata larga) → GND
Robot	placa + motores + controlador + sensores

Ejercicios

1. **Nombra tres diferencias entre Arduino UNO y micro:bit.**

2. **¿Qué hace este código: `digitalWrite(13, HIGH); delay(2000); digitalWrite(13, LOW); delay(2000);`?**

3. **Dibuja el circuito para encender un LED desde el pin 7.**

4. **¿Qué valor devolverá `analogRead` si la tensión es 2,5 V?**

5. **Modifica el «Blink» para que el LED parpadee 4 veces por segundo.**

6. **Escribe con bloques de MakeCode un termómetro que muestre la temperatura al pulsar A.**

7. **¿Por qué no se conecta un motor directamente a un pin de Arduino?**

8. **Explica qué es el hardware libre.**

9. **Diseña un juego de reflejos con micro:bit: gana quien pulse antes su botón cuando aparezca una imagen.**

10. **Describe un robot que esquive obstáculos: componentes y programa.**

11. **¿Qué pines de Arduino UNO admiten PWM y cómo se reconocen?**

12. **¿Qué rango de valores admite analogWrite? ¿Y qué devuelve analogRead?**

13. **¿Para qué sirve el monitor serie?**

14. **¿Qué diferencia hay entre un servomotor y un motor de corriente continua?**

15. **¿Qué deben tener en común dos micro:bit para comunicarse por radio?**

16. **¿Qué tipo de variable usarías en Arduino para guardar 23,5 °C? ¿Y para contar pulsaciones?**

17. **Escribe el código que lee un potenciómetro en A0 y regula el brillo de un LED en el pin 6.**

18. **Escribe el código que muestra en el monitor serie el valor de una LDR cada medio segundo.**

19. **Programa un zumbador en el pin 8 que toque do, re y mi (262, 294 y 330 Hz) durante 0,3 s cada una.**

20. **¿Por qué se usan constantes como const int LED_ROJO = 10;?**

21. **Haz que un servo barra de 0° a 180° de 10 en 10 grados, como un radar.**

22. **Explica qué es PWM con tus palabras.**

23. **Programa una votación con micro:bit: los alumnos pulsan A (sí) o B (no) y la placa central cuenta.**

24. **Encuentra el error: void loop() { digitalWrite(13, HIGH) delay(500); }**

25. **¿Qué es una biblioteca en Arduino? Pon un ejemplo.**

26. **Diseña una alarma de distancia: si el ultrasonidos mide menos de 20 cm, el zumbador pita más rápido cuanto más cerca está el objeto.**

27. **Programa una función parpadear(pin, veces) en Arduino y úsala para que el LED del pin 13 parpadee 3 veces al pulsar el botón del pin 2.**

28. **Diseña una red de sensores de temperatura con micro:bit en tres aulas y una placa central que muestre la más alta.**

29. **Compara usar delay() y millis() para que un LED parpadee mientras se lee un botón. ¿Por qué es mejor millis()?**

30. **Diseña un robot sigue-líneas con Arduino: componentes, conexiones y programa en pseudocódigo.**

Practica

1. **Un microcontrolador es...**

   A) un tipo de sensor
   B) un ordenador pequeño en un solo chip
   C) un cable USB
   D) una batería

2. **Arduino es un proyecto de...**

   A) software de pago
   B) una consola
   C) hardware libre
   D) una impresora

3. **En Arduino, la función que se ejecuta una sola vez es...**

   A) setup()
   B) loop()
   C) delay()
   D) pinMode()

4. **delay(1000) espera...**

   A) 1 minuto
   B) 1 milisegundo
   C) 1 segundo
   D) 1 hora

5. **analogRead devuelve en Arduino UNO un número entre...**

   A) 0 y 1
   B) 0 y 1023
   C) 0 y 100
   D) 0 y 255

6. **La resistencia en serie con un LED sirve para...**

   A) limitar la corriente
   B) aumentar la luz
   C) cambiar el color
   D) dar color

7. **La pata larga del LED es el...**

A) cátodo (–)
B) ánodo (+)
C) neutro
D) emisor

8. **¿Cuántos LED tiene la matriz de la micro:bit?**
   A) 16
   B) 64
   C) 25
   D) 100

9. **El editor web por bloques de la micro:bit es...**
   A) MakeCode
   B) Arduino IDE
   C) Photoshop
   D) Excel

10. **Para detectar que agitas la micro:bit se usa el...**
    A) sensor de luz
    B) acelerómetro
    C) botón B
    D) botón A

11. **En Arduino, las instrucciones terminan con...**
    A) un punto
    B) punto y coma
    C) dos puntos
    D) nada

12. **analogWrite admite valores de...**
    A) 0 a 1023
    B) 0 a 255
    C) 0 a 100
    D) 0 a 1

13. **Los pines PWM de Arduino UNO se marcan con...**
    A) *
    B) ~
    C) #

D) +

14. **Serial.begin(9600) sirve para...**

A) iniciar el monitor serie

B) encender un LED

C) mover un servo

D) leer un botón

15. **Un servomotor gira...**

A) sin parar

B) a un ángulo de 0 a 180°

C) solo con luz

D) a 1000 rpm siempre

16. **tone(8, 440, 500) toca...**

A) un la de 440 Hz medio segundo

B) 8 notas

C) durante 440 s

D) nada

17. **Para comunicarse por radio, dos micro:bit deben...**

A) tocarse

B) estar en el mismo grupo

C) tener el mismo programa

D) estar en el ordenador

18. **Para regular la velocidad de un motor se usa...**

A) digitalRead

B) PWM

C) Serial.println

D) delay

19. **Para controlar motores desde Arduino hace falta...**

A) un controlador de motores

B) una LDR

C) un zumbador

D) nada

20. **Una variable para guardar 21,7 °C en Arduino es de tipo...**

A) int

B) float

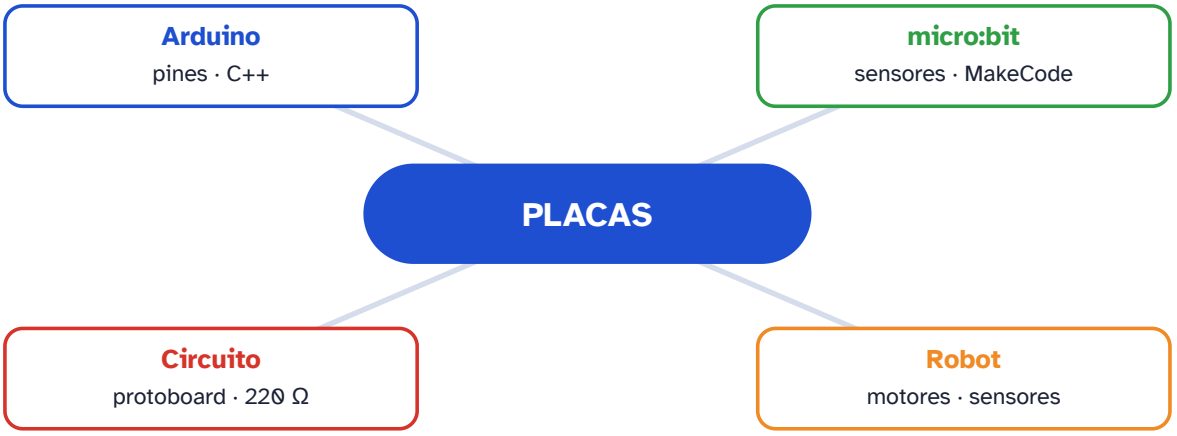
C) bool

D) char

Explícalo con tus palabras

Incluye: Qué es un microcontrolador y en qué se diferencia de un ordenador; Cómo se monta un LED con su resistencia; Qué hacen setup() y loop(); Ventajas de la micro:bit para empezar.

Resumen



Glosario

Microcontrolador	Ordenador pequeño en un solo chip.
Arduino	Placa de hardware libre para prototipos.
micro:bit	Placa educativa de la BBC.
Pin	Patilla de conexión de entrada o salida.
Protoboard	Placa para montar circuitos sin soldar.
Sketch	Programa de Arduino.
setup()	Función que se ejecuta una vez al empezar.
loop()	Función que se repite sin fin.
digitalWrite	Pone un pin en HIGH o LOW.
analogRead	Lee un valor de 0 a 1023.
PWM	Modulación por ancho de pulsos.
analogWrite	Salida PWM de 0 a 255.
Monitor serie	Comunicación de texto con el ordenador.
Servomotor	Motor que se coloca en un ángulo.
Biblioteca	Funciones ya programadas que se incluyen.
MakeCode	Editor por bloques de la micro:bit.
Hardware libre	Diseño electrónico con planos publicados.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Microcontrolador

Ordenador en un chip que ejecuta un programa sin parar.

Pin

Patilla de la placa para conectar entradas o salidas.

Protoboard

Placa para montar circuitos sin soldar.

setup()

Se ejecuta una vez al encender: configuración.

loop()

Se repite sin fin.

digitalWrite

Pone un pin a HIGH (5 V) o LOW (0 V).

analogRead

Lee una tensión como número de 0 a 1023.

MakeCode

Editor por bloques de la micro:bit con simulador.

PWM

Encender y apagar muy rápido para lograr valores intermedios.

Monitor serie

Ventana donde la placa envía mensajes al ordenador.

Servomotor

Motor que va a un ángulo de 0 a 180°.

Radio de micro:bit

Comunicación entre placas del mismo grupo.

Soluciones

1. **B)** Procesador, memoria y E/S.
2. **C)** Diseños públicos.
3. **A)** Configuración.
4. **C)** 1.000 ms.
5. **B)** 10 bits.
6. **A)** Evita que se queme.
7. **B)** Va hacia el positivo.
8. **C)** 5 × 5.
9. **A)** De Microsoft.
10. **B)** Movimiento e inclinación.
11. **B)** Punto y coma.
12. **B)** PWM de 8 bits.
13. **B)** El símbolo ~.
14. **A)** Comunicación con el ordenador.
15. **B)** Va a un ángulo y se queda.
16. **A)** Frecuencia y duración.
17. **B)** Mismo grupo de radio.
18. **B)** Potencia media.
19. **A)** Puente H.
20. **B)** Decimal.

Soluciones de los ejercicios

1. Por ejemplo: micro:bit tiene sensores integrados; Arduino necesita protoboard; micro:bit se programa con MakeCode y Arduino con su IDE en C++.
2. Enciende el LED del pin 13 dos segundos y lo apaga dos segundos, sin parar si está en loop().
3. Pin 7 → resistencia 220 Ω → pata larga del LED → pata corta → GND.
4. Aproximadamente 512 (la mitad de 1023).
5. Cada ciclo dura 250 ms: delay(125) encendido y delay(125) apagado.
6. al presionar botón A → mostrar número (temperatura (°C)).
7. Porque necesita más corriente de la que da un pin; se usa un controlador de motores y alimentación aparte.
8. Hardware cuyos diseños son públicos y pueden estudiarse, modificarse y fabricarse libremente.
9. Esperar un tiempo al azar; mostrar imagen; al presionar A → mostrar «A gana»; al presionar B → mostrar «B gana».
10. Placa, dos motores con controlador, batería, sensor de ultrasonidos; por siempre: si distancia < 15 cm, girar; si no, avanzar.
11. Los marcados con ~: 3, 5, 6, 9, 10 y 11.
12. analogWrite: 0 a 255. analogRead: 0 a 1023.
13. Para ver en el ordenador los mensajes y valores que envía la placa: calibrar sensores y depurar el programa.
14. El servo va a un ángulo concreto (0-180°) y se queda ahí; el motor de corriente continua gira sin parar.
15. Estar en el mismo grupo de radio.
16. float para 23,5; int para el contador.
17. int v = analogRead(A0); analogWrite(6, v / 4); (o map(v, 0, 1023, 0, 255)).
18. setup(): Serial.begin(9600); loop(): Serial.println(analogRead(A0)); delay(500);
19. tone(8, 262, 300); delay(350); tone(8, 294, 300); delay(350); tone(8, 330, 300); delay(350);
20. Para que el programa sea más legible y, si cambia la conexión, baste con cambiar una sola línea.
21. for (int a = 0; a <= 180; a += 10) { radar.write(a); delay(200); } y después el bucle inverso de 180 a 0.
22. Encender y apagar el pin muy rápido; según el porcentaje de tiempo encendido, el LED o el motor reciben una potencia media mayor o menor y parecen brillar o girar a medias.
23. Alumnos: al presionar A: radio enviar número 1; B: enviar 0. Central: al recibir n: si n = 1: síes + 1; si no: noes + 1. Al presionar A+B: mostrar síes y noes.
24. Falta el punto y coma tras digitalWrite(13, HIGH). Además, el LED nunca se apaga: falta digitalWrite(13, LOW); delay(500);
25. Un conjunto de funciones ya programadas que se añaden con #include, como Servo.h para controlar servomotores.

26. Medir d. Si d < 20: tone(8, 1000, 50); delay(d · 10); (a menos distancia, menos espera entre pitidos). Si no: noTone(8). Mostrar d en el monitor serie para calibrar.
27. void parpadear(int pin, int veces) { for (int i = 0; i < veces; i++) { digitalWrite(pin, HIGH); delay(200); digitalWrite(pin, LOW); delay(200); } } En loop(): if (digitalRead(2) == LOW) parpadear(13, 3);
28. Cada placa de aula: cada minuto, radio enviar valor «aula1», temperatura. Central: al recibir nombre y valor: guardar en variables por aula; mostrar el máximo y, si supera 26 °C, mostrar un aviso. Todas en el mismo grupo.
29. delay() detiene el programa: durante la espera no se lee el botón. Con millis() se comprueba cuánto tiempo ha pasado y se cambia el LED solo cuando toca, sin detenerse, así el botón se lee siempre.
30. Arduino, controlador L298N, dos motores con ruedas, dos sensores IR, batería. Sensores en pines 2 y 3; motores en 5, 6 (PWM) y 7, 8. Programa: leer izq y der; ambos blanco: avanzar; izq negro: girar izquierda; der negro: girar derecha; ambos negro: parar.